

测试用例清单

项目：企业级智能运营中枢（AI Ops Hub）—— ai-ops-full

版本：V1.0 测试类型：单元 / 集成 / API / 前端 / 端到端

覆盖目标: LangChain4j 全部核心功能模块 (ReAct / Plan-and-Execute / Supervisor 多 Agent、RAG+ReRank+混合检索、多 Tool、结构化输出、InMemory/Redis 记忆、SSE 流式、审计与安全合规)

一、测试策略与分层

| 层级 | 目录 / 工具 | 覆盖范围 | 运行方式 |

| 后端单元测试 | src/test/java/... (JUnit5 + Mockito) | Tool 方法、RAG 检索、POJO、配置 | mvn test |

```
| 后端集成测试 | src/test/java/... (Spring Boot Test + WebClient) | REST API、Agent 端到端、RAG 摄取+检索
| mvn verify |
```

```
| 前端组件测试 | frontend/src/**/__tests__/*.spec.js (Vitest + Vue Test Utils) | 页面渲染、事件交互、API 调用
| npm run test |
```

|端到端测试|手动 / Postman 集合|全链路对话、工具调用、结构化报告|见第四节|

二、后端测试用例明细

2.1 对话记忆 (ChatMemoryConfigTest)

用例编号	场景	输入	预期结果	优先级
------	----	----	------	-----

MEM-01	Spring 容器加载配置类	启动上下文 (in-memory 模式)	ChatMemoryConfig 被成功加载, 不为 null	P0
--------	----------------	----------------------	---------------------------------	----

MEM-02 | ChatMemory Bean 类型校验 | 获取 chatMemory Bean | 返回 MessageWindowChatMemory 实例 | P0 |

MEM-03	窗口大小符合配置	默认配置	maxMessages == 20	P1
--------	----------	------	-------------------	----

2.2 RAG 检索服务 (RagServiceTest, Mock EmbeddingStore)

用例编号	场景	输入	预期结果	优先级
------	----	----	------	-----

RAG-01	无匹配时返回空串	任意查询, store 返回空列表	返回 "", 不抛异常	P0
--------	----------	-------------------	-------------	----

RAG-02	多片段召回拼接	查询, store 返回 2 个匹配片段	返回文本同时包含两段内容	P0
--------	---------	----------------------	--------------	----

| RAG-03 | topN 参数链路通畅 | 查询 | findRelevant 以 topN=3 调用, 不抛异常 | P1 |

2.3 RAG 端到端 (RagIngestionTest)

用例编号	场景	输入	预期结果	优先级
------	----	----	------	-----

| RAG-E2E-01 | 文档摄取后语义检索可召回 | 写入 ops-guide.md, 查询"连接池耗尽怎么排查"

召回片段文本包含"连接池" | P0 |

2.4 工具集 (OpsAgentToolsTest)

用例编号	场景	输入	预期结果	优先级
------	----	----	------	-----

TOL-01	知识库检索委托 RagService	关键词"DB 连接超时"	返回 RagService 对应内容	P0
TOL-02	空关键词检索不抛异常	""	返回 ""	P2
TOL-03	日志查询返回非空列表	serviceName, minutes	列表非空	P0
TOL-04	日志包含 ERROR/WARN 级别	order-service, 30	结果含 ERROR 与 WARN	P1
TOL-05	日志仅包含指定服务名	payment-service, 10	全部条目含"payment-service"	P2
TOL-06	创建工单返回标准格式	标题/优先级/描述	含"TICKET-"前缀	P0
TOL-07	两次工单号不同 (UUID 随机)	两次不同调用	两个 ticketId 不相等	P2

2.5 结构化输出 POJO (ReportTest)

用例编号	场景	输入	预期结果	优先级
POJO-01	Record 字段正确持有值	new Report(...)	各 accessor 返回传入值	P0
POJO-02	equals/hashCode 基于值	两个相同值 Record	equals true、hashCode 相等	P1
POJO-03	toString 包含各字段	new Report(...)	字符串含所有字段值	P2

2.6 REST API 集成 (ChatControllerTest, Mock Agent + WebTestClient)

用例编号	场景	请求	预期结果	优先级
API-01	/ai/analyze 返回结构化报告	POST 事件描述	200, JSON 含 rootCause/impact/suggestion/ticketId	P0
API-02	/ai/analyze 空 body 校验失败	POST ""	400 Bad Request	P1
API-03	/ai/chat 缺少 userId 参数	POST 描述	400 Bad Request	P1

三、前端测试用例明细 (Vitest)

3.1 知识库页面 (KnowledgeView.spec.js)

用例编号	场景	预期结果	优先级
FE-KB-01	初始文档列表为空	documents == []	P1
FE-KB-02	目录为空时不调用摄取接口	axios.post 未被调用	P1
FE-KB-03	填写目录后调用摄取接口	请求 URL 含 /ai/rag/ingest	P0
FE-KB-04	检索后结果列表被正确填充	results.length == 1, 文本匹配	P0

3.2 事件分析页面 (IncidentView.spec.js)

用例编号	场景	预期结果	优先级
FE-INC-01	提交事件后渲染结构化报告	report 非空, 字段映射正确	P0
FE-INC-02	空描述不调用分析接口	axios.post 未被调用	P1

3.3 API 服务层 (api.spec.js)

用例编号	场景	预期结果	优先级
FE-API-01	analyzeIncident POST 到正确端点	URL 含 /ai/analyze?userId=	P0

| FE-API-02 | ingestDocuments POST 到正确端点 | URL 含 /ai/rag/ingest, body 含 dir | P1 |

四、端到端 / 接口验收用例（手工 / Postman）

| 用例编号 | 场景 | 步骤 | 预期结果 | 优先级 |

---|---|---|---|---

| E2E-01 | 流式对话 + 工具调用 | POST /ai/chat?userId=ops-001, body 含"order-service 500" | SSE 流式返回, Agent 自主调用日志/知识库/建工单工具 | P0 |

| E2E-02 | Supervisor 路由到 invest Agent | POST /ai/chat, body 含"评估投入 500 万做新产品线" | 路由到投资决策子 Agent, 返回含 IRR/建议的结构化内容 | P0 |

| E2E-03 | Plan-and-Execute 投资决策 | POST /ai/invest/advise?project=新产品线 | 先规划再分步调 IRR 工具, 返回 InvestmentAdvice | P0 |

| E2E-04 | RAG 文档摄取 + 检索 | POST /ai/rag/ingest?dir=/path/to/docs, 随后查询 | 文档被摄取并可被语义检索召回 | P0 |

| E2E-05 | 多用户记忆隔离 | 分别用 userId=A/B 对话后切换 | A/B 各自记忆独立, 不串话 | P1 |

| E2E-06 | Redis 记忆持久化 (需 Redis) | application.yml 切 redis, 重启后继续会话 | 历史上下文可恢复 | P2 |

五、性能 / 安全 / 合规测试要点

| 类型 | 用例 | 方法 | 验收标准 |

---|---|---|---

| 性能 | 单次对话首字节延迟 | 流式接口压测 (并发 10) | P95 首字节 < 3s |

| 性能 | 工具调用超时兜底 | Tool 执行超 30s | Agent 返回友好提示并继续, 不崩溃 |

| 安全 | 内容审核 | 输入含敏感词 | SecurityTools 审核拦截并返回风险等级 |

| 安全 | 审计完整性 | 执行任意 Agent 调用 | AuditAspect 记录耗时/状态/入参脱敏 |

| 合规 | 引用溯源 | RAG 回答 | 输出标注来源文档, 可点击查看 |

六、测试执行与报告

后端

```
cd ai-ops-agent
export OPENAI_API_KEY=sk-xxx # API ████████████████████/██████ Mock██████
./mvnw test # ██████████
./mvnw verify # █████ + █████
```

前端

```
cd frontend
npm install
npm run test -- --run # █ Vitest ██████████
npm run test # █████
```

测试报告输出

- 后端 Surefire 报告: ai-ops-agent/target/surefire-reports/
- 前端 Vitest 报告: 在 frontend 下配置 --reporter=json --outputFile=test-results.json 可输出 JSON 报告

七、实现状态对照

LangChain4j 能力	对应测试	状态
ReAct / Tool Calling	TOL-01~07、E2E-01	☒ 已实现并覆盖
RAG 全链路（摄取+检索）	RAG-01~03、RAG-E2E-01、FE-KB-01~04	☒ 已实现并覆盖
结构化输出	POJO-01~03、API-01、FE-INC-01	☒ 已实现并覆盖
对话记忆	MEM-01~03、E2E-05~06	☒ 已实现（E2E-06 需 Redis）
Plan-and-Execute	E2E-03	☒ 端到端覆盖
Supervisor 多 Agent	E2E-02	☒ 端到端覆盖
SSE 流式	E2E-01	☒ 端到端覆盖
审计与安全合规	安全/合规要点	☒ AuditAspect + SecurityTools 已落地
MCP 协议 / 多厂商网关	—	☒ V2.0/V3.0 迭代

本文档与 ai-ops-full 项目源码配套：后端测试位于 ai-ops-agent/src/test/，前端测试位于 frontend/src/**/__tests__/ 与 frontend/src/services/__tests__/。所有单元测试与集成测试不依赖外部中间件（Redis/Milvus 为可选，默认 in-memory），开箱可运行。